

ALGORITHMS AND TURING MACHINES

BACKGROUND TO THE ALGORITHM CONCEPT

WHAT PRECISELY IS an algorithm, or a Turing machine, or a universal Turing machine? Why should these concepts be so central to the modern view of what could constitute a 'thinking device'? Are there any absolute limitations to what an algorithm could in principle achieve? In order to address these questions adequately, we shall need to examine the idea of an algorithm and of Turing machines in some detail.

In the various discussions which follow, I shall sometimes need to refer to mathematical expressions. I appreciate that some readers may be put off by such things, or perhaps find them intimidating. If you are such a reader, I ask your indulgence, and recommend that you follow the advice I have given in my 'Note to the reader' on p. viii! The arguments given here do not require mathematical knowledge beyond that of elementary school, but to follow them in detail, some serious thought would be required. In fact, most of the descriptions are quite explicit, and a good understanding can be obtained by following the details. But much can also be gained even if one simply skims over the arguments in order to obtain merely their flavour. If, on the other hand, you are an expert, I again ask your indulgence. I suspect that it may still be worth your while to look through what I have to say, and there may indeed be a thing or two to catch your interest.

The word 'algorithm' comes from the name of the ninth century Persian mathematician Abu Ja'far Mohammed ibn Mûsâ al-Khowârizm who wrote an influential mathematical textbook, in

about 825 AD, entitled 'Kitab al-jabr wa'l-muqabala'. The way that the name 'algorithm' has now come to be spelt, rather than the earlier and more accurate 'algorism', seems to have been due to an association with the word 'arithmetic'. (It is noteworthy, also, that the word 'algebra' comes from the Arabic 'al-jabr' appearing in the title of his book.)

Instances of algorithms were, however, known very much earlier than al-Khowârizm's book. One of the most familiar, dating from ancient Greek times (c. 300 BC), is the procedure now referred to as *Euclid's algorithm* for finding the highest common factor of two numbers. Let us see how this works. It will be helpful to have a specific pair of numbers in mind, say 1365 and 3654. The highest common factor is the largest single whole number that divides into each of these two numbers exactly. To apply Euclid's algorithm, we divide one of our two numbers by the other and take the remainder: 1365 goes twice into 3654, with remainder 924 ($= 3654 - 2730$). We now replace our original two numbers by this remainder, namely 924, and the number we just divided by, namely 1365, in that order. We repeat what we just did using this new pair: 924 goes once into 1365, with remainder 441. This gives another new pair, 441 and 924, and we divide 441 into 924 to get the remainder 42 ($= 924 - 882$), and so on until we get a division that goes exactly. Setting all this out, we get:

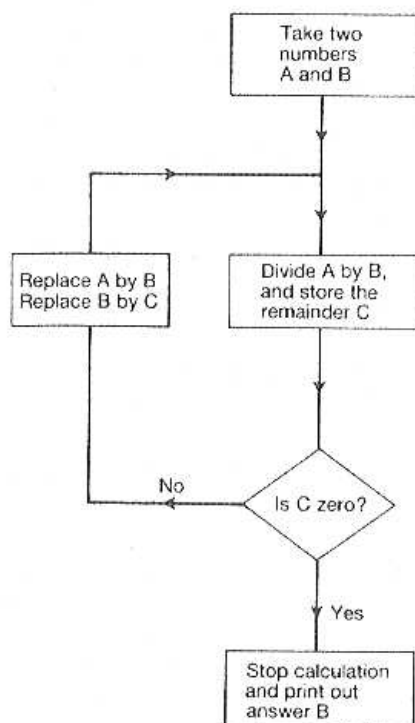
$$\begin{array}{ll} 3654 \div 1365 & \text{gives remainder } 924 \\ 1365 \div 924 & \text{gives remainder } 441 \\ 924 \div 441 & \text{gives remainder } 42 \\ 441 \div 42 & \text{gives remainder } 21 \\ 42 \div 21 & \text{gives remainder } 0 \end{array}$$

The last number we divided by, namely 21, is the required highest common factor.

Euclid's algorithm itself is the *systematic procedure* by which we found this factor. We have just applied this procedure to a particular pair of numbers, but the procedure itself applies quite generally, to numbers of any size. For very large numbers, the procedure could take a very long time to carry out, and the larger the numbers, the longer the procedure will tend to take. But in any *specific* case the procedure will eventually terminate and a definite

answer will be obtained in a finite number of steps. At each step it is perfectly clear-cut what the operation is that has to be performed, and the decision as to the moment at which the whole process has terminated is also perfectly clear-cut. Moreover, the description of the whole procedure can be presented in *finite* terms, despite the fact that it applies to natural numbers of unlimited size. (The 'natural numbers', are simply the ordinary non-negative¹ whole numbers 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...). Indeed, it is easy to construct a (finite) 'flow chart' to describe the full logical operation of Euclid's algorithm (see below).

It should be noted that this procedure is still not quite broken down into its most elementary parts since it has been implicitly assumed that we already 'know' how to perform the necessary basic operation of obtaining the remainder from a division, for



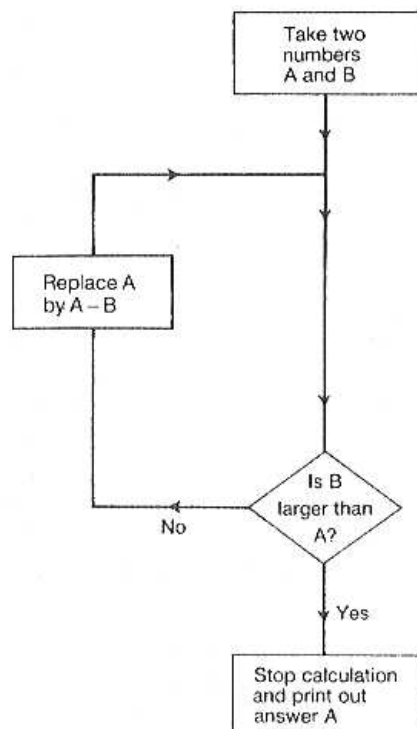
two arbitrary natural numbers A and B. That operation is again algorithmic – performed by the very familiar procedure for division that we learn at school. That procedure is actually rather more complicated than the rest of Euclid's algorithm, but again a flow chart may be constructed. The main complication results from the fact that we would (presumably) be using the standard 'denary' notation for natural numbers, so that we would need to list all our multiplication tables and worry about carrying, etc. If we had simply used a succession of n marks of some kind to represent the number n – for example, ●●●●● to represent five – then the forming of a remainder would be seen as a very elementary algorithmic operation. In order to obtain the remainder when A is divided by B we simply keep removing the succession of marks representing B from that representing A until there are not enough marks left to perform the operation again. The last remaining succession of marks provides the required answer. For example, to obtain the remainder when seventeen is divided by five we simply proceed by removing successions of ●●●●● from ●●●●●●●●●●●●●●● as follows:

●●●●●●●●●●●●●●●
 ●●●●●●●●●●
 ●●●●●●●●
 ●●

and the answer is clearly two since we cannot perform the operation again.

A flow chart which finds the remainder in a division, achieved by this means of repeated subtraction, given as on p. 44. To complete the entire flow chart for Euclid's algorithm, we substitute this chart for forming a remainder into the box at the centre right in our original chart. This kind of substitution of one algorithm into another is a common computer-programming procedure. The algorithm on p. 44 for finding a remainder is an example of a *subroutine*, that is, it is a (normally previously known) algorithm which is called upon and used by the main algorithm as a part of its operation.

Of course the representation of the number n simply as n spots is very inefficient when large numbers are involved, which is why



we normally use a more compact notation such as the standard (denary) system. However we shall not be much concerned with the *efficiency* of operations or notations here. We are concerned instead with the question of what operations can *in principle* be performed algorithmically. What is algorithmic if we use one notation for numbers is also algorithmic if we use the other. The only differences lie in the detail and complication in the two cases.

Euclid's algorithm is just one among the numerous, often classical, algorithmic procedures that are to be found throughout mathematics. But it is perhaps remarkable that despite the ancient historic origins of specific examples of algorithms, the precise formulation of the *concept of a general algorithm* dates only from *this* century. In fact, various alternative descriptions of this concept have been given, all in the 1930s. The most direct and

persuasive of these, and also historically the most important, is in terms of the concept known as a *Turing machine*. It will be appropriate for us to examine these 'machines' in some detail.

One thing to bear in mind about a Turing 'machine' will be that it is a piece of 'abstract mathematics' and not a physical object. The concept was introduced by the English mathematician, code-breaker extraordinary and seminal computer scientist Alan Turing in 1935–6 (Turing 1937), in order to tackle a very broad-ranging problem, known as the *Entscheidungsproblem*, partially posed by the great German mathematician David Hilbert in 1900, at the Paris International Congress of Mathematicians ('Hilbert's tenth problem'), and more completely, at the Bologna International Congress in 1928. Hilbert had asked for no less than a general algorithmic procedure for resolving mathematical questions – or, rather, for an answer to the question of whether or not such a procedure might in principle exist. Hilbert also had a programme for placing mathematics on an unassailably sound foundation, with axioms and rules of procedure which were to be laid down once and for all, but by the time Turing produced his great work, that programme had already suffered a crushing blow from a startling theorem proved in 1931 by the brilliant Austrian logician Kurt Gödel. We shall be considering the Gödel theorem and its significance in Chapter 4. The problem of Hilbert's that concerned Turing (the *Entscheidungsproblem*) went beyond any particular formulation of mathematics in terms of axiomatic systems. The question was: is there some general mechanical procedure which could, *in principle*, solve all the problems of mathematics (belonging to some suitably well-defined class) one after the other?

Part of the difficulty in answering this question was to decide what is to be meant by a 'mechanical procedure'. The concept lay outside the normal mathematical ideas of the time. In order to come to grips with it, Turing tried to imagine how the concept of a 'machine' could be formalized, its operation being broken down into elementary terms. It seems clear that Turing also regarded a human brain to be an example of a 'machine' in his sense, so whatever the activities might be that are carried out by human mathematicians when they tackle their problems of mathematics,

these also would have to come under the heading of 'mechanical procedures'.

Whilst this view of human thinking appears to have been valuable to Turing in the development of his highly important concept, it is by no means necessary for us to adhere to it. Indeed, by making precise what is meant by a mechanical procedure, Turing actually showed that there are some perfectly well-defined mathematical operations which cannot, in any ordinary sense, be called mechanical! There is perhaps some irony in the fact that this aspect of Turing's own work may now indirectly provide us with a possible loophole to his own viewpoint concerning the nature of mental phenomena. However, this is not our concern for the moment. We need first to find out what Turing's concept of a mechanical procedure actually is.

TURING'S CONCEPT

Let us try to imagine a device for carrying out some (finitely definable) calculational procedure. What general form would such a device take? We must be prepared to idealize a little and not worry too much about practicalities: we are really thinking of a mathematically idealized 'machine'. We want our device to have a discrete set of different possible states, which are *finite* in number (though perhaps a very large number). We call these the *internal* states of the device. However we do not want to limit the size of the calculations that our device will perform in principle. Recall Euclid's algorithm described above. There is, in principle, no limit to the size of the numbers on which that algorithm can act. The algorithm – or the general calculational *procedure* – is just the same no matter how large the numbers are. For very large numbers, the procedure may indeed take a very long time, and a considerable amount of 'rough paper' may be needed on which to perform the actual calculations. But the *algorithm* is the same *finite* set of instructions no matter how big the numbers.

Thus, although it has a finite number of internal states, our device must be able to deal with an input which is not restricted in its size. Moreover, the device must be allowed to call upon an

unlimited external storage space (our 'rough paper') for its calculations, and also to be able to produce an output of unlimited size. Since our device has only a finite number of distinct internal states, it cannot be expected to 'internalize' all the external data nor all the results of its own calculations. Instead it must examine only those parts of the data or previous calculations that it is *immediately* dealing with, and then perform whatever operation it is required to perform on them. It can note down, perhaps in the external storage space, the relevant results of that operation, and then proceed in a precisely determined way to the next stage of operation. It is the unlimited nature of the input, calculation space, and output which tells us that we are considering only a mathematical idealization rather than something that could be actually constructed in practice (see Fig. 2.1). But it is an idealization of great relevance. The marvels of modern computer technology have provided us with electronic storage devices which can, indeed, be treated as unlimited for most practical purposes.

In fact, the type of storage space that has been referred to as 'external' in the above discussion could be regarded as actually part of the internal workings of a modern computer. It is perhaps a technicality whether a certain part of the storage space is to be regarded as internal or external. One way of referring to this division between the 'device' and the 'external' part would be in terms of *hardware* and *software*. The internal part could then be

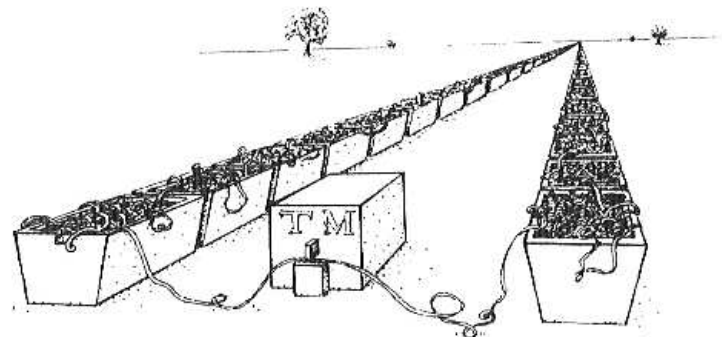


Fig. 2.1. A strict Turing machine requires an infinite tape!

the hardware and the external part, the software. I shall *not* necessarily stick to this, but whichever way one looks at it, Turing's idealization is indeed remarkably well approximated by the electronic computers of today.

The way that Turing pictured the external data and storage space was in terms of a 'tape' with marks on it. This tape would be called upon by the device and 'read' as necessary, and the tape could be moved backwards or forwards by the device, as part of its operation. The device could also place new marks on the tape where required, and could obliterate old ones, allowing the *same* tape to act as external storage (i.e. 'rough paper') as well as input. In fact it is helpful not to make any clear distinction between 'external storage' and 'input' because in many operations the intermediate results of a calculation can play a role just like that of new data. Recall that with Euclid's algorithm we kept replacing our original input (the numbers A and B) by the results of the different stages of the calculation. Likewise, the same tape can be used for the final output (i.e. the 'answer'). The tape will keep running back and forth through the device so long as further calculations need to be performed. When the calculation is finally completed, the device comes to a halt and the answer to the calculation is displayed on that part of the tape which lies to one side of the device. For definiteness, let us suppose that the answer is always displayed on the left, while all the numerical data in the input, together with the specification of the problem to be solved, always goes in on the right.

For my own part, I feel a little uncomfortable about having our finite device moving a potentially infinite tape backwards and forwards. No matter how lightweight its material, an *infinite* tape might be hard to shift! Instead, I prefer to think of the tape as representing some external environment through which our finite device can move. (Of course, with modern electronics, neither the 'tape' nor the 'device' need actually 'move' in the ordinary physical sense, but such 'movement' is a convenient way of picturing things.) On this view, the device receives all its input from this environment. It uses the environment as its 'rough paper'. Finally it writes out its output on this same environment.

In Turing's picture the 'tape' consists of a linear sequence of

squares, which is taken to be infinite in both directions. Each square on the tape is either blank or contains a single mark.* The use of marked or unmarked squares illustrates that we are allowing our 'environment' (i.e. the tape) to be broken down and described in terms of *discrete* (as opposed to continuous) elements. This seems to be a reasonable thing to do if we wish our device to function in a reliable and absolutely definite way. We are, however, allowing this 'environment' to be (potentially) infinite, as a feature of the mathematical idealization that we are using, but in any *particular* case the input, calculation and output must always be *finite*. Thus, although the tape is taken to be infinitely long, there must only be a finite number of actual marks on it. Beyond a certain point in each direction the tape must be entirely blank.

We indicate a blank square by the symbol '0' and a marked one by the symbol '1', e.g.:

0	0	0	1	1	1	1	0	1	0	0	1	1	1	0	0	1	0	0	1	0	1	1	0	1	0	0	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

We need our device to 'read' the tape, and we shall suppose that it does this *one* square at a time, and after each operation moves just *one* square to the right or left. There is no loss in generality involved in that. A device which reads n squares at a time or moves k squares at a time can easily be modelled by another device which reads and moves just one square at a time. A movement of k squares can be built up of k movements of one square, and by storing up n readings of one square it can behave as though it reads all n squares at once.

What, in detail, can such a device do? What is the most general way in which something that we would describe as 'mechanical' might function? Recall that the *internal states* of our device are to be finite in number. All we need to know, beyond this finiteness is that the behaviour of the device is completely determined by its internal state and by the input. This input we have simplified to

* In fact, in his original descriptions Turing allowed his tape to be marked in more complicated ways, but this makes no real difference. The more complicated marks could always be broken down into successions of marks and blanks. I shall be taking various other unimportant liberties with Turing's original specifications.

being just one of the two symbols '0' or '1'. Given its initial state and this input, the device is to operate completely deterministically: it changes its internal state to some other (or possibly the same) internal state; it replaces the 0 or 1 that it is just reading by the same or different symbol 0 or 1; it moves one square either to the right or left; finally, it decides whether to continue the calculation or to terminate it and come to a halt.

To define the operation of our device in an explicit way, let us first *number* the different internal states, say by the labels 0, 1, 2, 3, 4, 5, . . . ; the operation of the device, or *Turing machine*, would then be completely specified by some explicit list of replacements, such as:

00 → 00R
 01 → 131L
 10 → 651R
 11 → 10R
 20 → 01R.STOP
 21 → 661L
 30 → 370R
 . . .
 2100 → 31L
 . . .
 2581 → 00R.STOP
 2590 → 971R
 2591 → 00R.STOP

The *large figure* on the left-hand side of the arrow is the symbol on the tape that the device is in the process of reading, and the device replaces it by the large figure at the middle on the right. R tells us that the device is to move one square to the *right* along the tape and L tells us that it is to move by one step to the *left*. (If, as with Turing's original descriptions, we think of the tape moving rather than the device, then we must interpret R as the instruction to move the *tape* one square to the *left* and L as moving it one square

to the *right*.) The word STOP indicates that the calculation has been completed and the device is to come to a halt. In particular, the second instruction 01 → 131L tells us that *if* the device is in internal state 0 and reads 1 on the tape then it must change to internal state 13, leave the 1 as a 1 on the tape, and move one square along the tape to the left. The last instruction 2591 → 00R.STOP tells us that if the device is in state 259 and reads 1 on the tape, then it must revert to state 0, erase the 1 to produce 0 on the tape, move one square along the tape to the right, and terminate the calculation.

Instead of using the numerals 0, 1, 2, 3, 4, 5, . . . for labelling the internal states, it would be somewhat more in keeping with the above notation for marks on the tape if we were to use symbols made up of just 0s and 1s. We could simply use a succession of *n* 1s to label the state *n* if we choose, but that is inefficient. Instead, let us use the *binary* numbering system, which is now a familiar mode of notation:

0 → 0,
 1 → 1,
 2 → 10,
 3 → 11,
 4 → 100,
 5 → 101,
 6 → 110,
 7 → 111,
 8 → 1000,
 9 → 1001,
 10 → 1010,
 11 → 1011,
 12 → 1100, etc.

Here the final digit on the right refers to the 'units' just as it does in the standard (denary) notation, but the digit just before it refers to 'twos' rather than 'tens'. The one before that refers to 'fours' rather than 'hundreds' and before that, to 'eights' rather than 'thousands', and so on, the value of each successive digit, as we move to the left, being the successive *powers of two*: 1, 2, 4 ($= 2 \times 2$), 8 ($= 2 \times 2 \times 2$), 16 ($= 2 \times 2 \times 2 \times 2$), 32 ($= 2 \times 2 \times 2 \times 2 \times 2$),

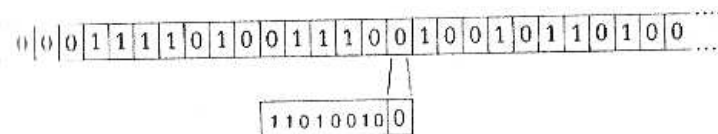
etc. (For some other purposes that we shall come to later, we shall also sometimes find it useful to use a base other than two or ten to represent natural numbers: e.g. in base *three*, the denary number 64 would be written 2101, each digit having a value which is now a power of three: $64 = (2 \times 3^3) + 3^2 + 1$; cf. Chapter 4, p. 138, footnote.)

Using such a binary notation for the internal states, the specification of the above Turing machine would now be:

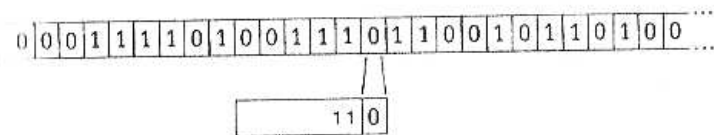
00	→	00R
01	→	11011L
10	→	10000011R
11	→	10R
100	→	01STOP
101	→	10000101L
110	→	1001010R
.		.
.		.
.		.
110100100	→	111L
.		.
.		.
.		.
.		.
1000000101	→	00STOP
1000000110	→	11000011R
1000000111	→	00STOP

In the above, I have also abbreviated R.STOP to STOP, since we may as well assume that L.STOP never occurs so that the result of the final step in the calculation is always displayed at the left of the device, as part of the answer.

Let us suppose that our device is in the particular internal state represented by the binary sequence 11010010 and is in the midst of a calculation for which the tape is given as on p. 49, and we apply the instruction $11010010 \rightarrow 111L$. The particular digit on the tape that is being read (here the digit '0') is indicated by a larger figure, to the right of the string of symbols representing the internal state. In the example of a Turing machine as partly specified above (and



which I have made up more or less at random), the '0' which is being read would be replaced by a '1' and the internal state would be changed to '11'; then the device would be moved one step to the left:



The device is now ready to read another digit, again a '0'. According to the table, it now leaves this '0' unchanged, but replaces the internal state by '100101' and moves back along the tape to the right by one step. Now it reads '1', and somewhere down the table would be a further instruction as to what replacement to make in its internal state, whether it should change the digit it is reading, and in which direction it should move along the tape. It would continue this way until it reaches a STOP, at which point (after it moves one further step to the right) we imagine a bell ringing to alert the operator of the machine that the calculation has been completed.

We shall suppose that the machine is always started with internal state '0' and that all the tape to the left of the reading device is initially blank. The instructions and data are all fed in at the right. As mentioned earlier, this information which is fed in is always to take the form of a *finite* string of 0s and 1s, followed by blank tape (i.e. 0s). When the machine reaches STOP, the result of the calculation appears on the tape to the left of the reading device.

Since we wish to be able to include numerical data as part of our input, we shall want to have a way of describing ordinary numbers (by which I here mean the natural numbers 0, 1, 2, 3, 4, . . .) as part of the input. One way to do this might be simply to

use a string of n 1s to represent the number n (although this could give us a difficulty with the natural number zero):

$1 \rightarrow 1, 2 \rightarrow 11, 3 \rightarrow 111, 4 \rightarrow 1111, 5 \rightarrow 11111$, etc.

This primitive numbering system is referred to (rather illogically) as the *unary* system. Then the symbol '0' could be used as a space to separate different numbers from one another. It is important that we have such a means of separating numbers from one another since many algorithms act on *sets* of numbers rather than on just single numbers. For example, for Euclid's algorithm, our device would need to act on the *pair* of numbers A and B. Turing machines can be written down, without great difficulty, which effect this algorithm. As an exercise, some dedicated readers might perhaps care to verify that the following explicit description of a Turing machine (which I shall call EUC) does indeed effect Euclid's algorithm when applied to a pair of unary numbers separated by a 0:

00 $\rightarrow$ 00R, 01 $\rightarrow$ 11L, 10 $\rightarrow$ 101R, 11 $\rightarrow$ 11L,
100 $\rightarrow$ 1010OR, 101 $\rightarrow$ 11OR, 110 $\rightarrow$ 100OR, 111 $\rightarrow$ 111R,
1000 $\rightarrow$ 1000OR, 1001 $\rightarrow$ 101OR, 1010 $\rightarrow$ 111OL,
1011 $\rightarrow$ 1101L, 1100 $\rightarrow$ 110OL, 1101 $\rightarrow$ 11L,
1110 $\rightarrow$ 111OL, 1111 $\rightarrow$ 10001L, 10000 $\rightarrow$ 1001OL,
10001 $\rightarrow$ 10001L, 10010 $\rightarrow$ 10OR, 10011 $\rightarrow$ 11L,
10100 $\rightarrow$ 00STOP, 10101 $\rightarrow$ 10101R.

Before embarking on this, however, it would be wise for any such reader to start with something much simpler, such as the Turing machine UN + 1:

00 $\rightarrow$ 00R, 01 $\rightarrow$ 11R, 10 $\rightarrow$ 01STOP, 11 $\rightarrow$ 11R,

which simply adds one to a unary number. To check that UN + 1 does just that, let us imagine that it is applied to, say, the tape

... 00000111100000 ... ,

which represents the number 4. We take the device to be initially somewhere off to the left of the 1s. It is in internal state 0 and reads a 0. This it leaves as 0, according to the first instruction, and it moves off one step to the right, staying in internal state 0. It keeps

doing this, moving one step to the right until it meets the first 1. Then the second instruction comes into play: it leaves the 1 as a 1 and moves to the right again, but now in internal state 1. In accordance with the fourth instruction, it stays in internal state 1, leaving the 1s alone, moving along to the right until it reaches the first 0 following the 1s. The third instruction then tells it to change that 0 to a 1, move one further step to the right (recall that STOP stands for R.STOP) and then halt. Thus, another 1 has been added to the string of 1s, and the 4 of our example has indeed been changed to 5, as required.

As a somewhat more taxing exercise, one may check that the machine UN $\times$ 2, defined by

00 $\rightarrow$ 00R, 01 $\rightarrow$ 10R, 10 $\rightarrow$ 101L, 11 $\rightarrow$ 11R,
100 $\rightarrow$ 110R, 101 $\rightarrow$ 100OR, 110 $\rightarrow$ 01STOP, 111 $\rightarrow$ 111R,
1000 $\rightarrow$ 1011L, 1001 $\rightarrow$ 1001R, 1010 $\rightarrow$ 101L,
1011 $\rightarrow$ 1011L,

doubles a unary number, as it is intended to.

In the case of EUC, to get the idea of what is involved, some suitable explicit pair of numbers can be tried, say 6 and 8. The reading device is, as before, taken to be in state 0 and initially on the left, and the tape would now be initially marked as:

... 0000000000011111101111111100000 ...

After the Turing machine comes to a halt, many steps later, we get a tape marked

... 000011000000000000 ...

with the reading device to the right of the non-zero digits. Thus the required highest common factor is (correctly) given as 2.

The full explanation of *why* EUC (or, indeed, UN $\times$ 2) actually does what it is supposed to do involves some subtleties and would be rather more complicated to explain than the machine is complicated itself – a not uncommon feature of computer programs! (To understand fully why an algorithmic procedure does what it is supposed to involves *insights*. Are 'insights' themselves algorithmic? This is a question that will have importance for us later.) I shall not attempt to provide such an explanation here for

the examples EUC or $UN \times 2$. The reader who does check them through will find that I have taken a very slight liberty with Euclid's actual algorithm in order to express things more concisely in the required scheme. The description of EUC is still somewhat complicated, comprising 22 elementary instructions for 11 distinct internal states. Most of the complication is of a purely organizational kind. It will be observed, for example, that of the 22 instructions, only 3 actually involve altering marks on the tape! (Even for $UN \times 2$ I have used 12 instructions, half of which involve altering the marks.)

BINARY CODING OF NUMERICAL DATA

The unary system is exceedingly inefficient for the representation of numbers of large size. Accordingly, we shall usually use the *binary* number system, as described earlier. However, we cannot just do this directly, attempting to read the tape simply as a binary number. As things stand, there would be no way of telling when the binary representation of the number has come to an end and the infinite succession of 0s representing the blank tape on the right begins. We need some notation for terminating the binary description of a number. Moreover, we shall often want to feed in *several* numbers, as with the *pair* of numbers² required for Euclid's algorithm. As things stand, we cannot distinguish the *spaces* between numbers from the 0s or strings of 0s that appear as parts of the binary representation of single numbers. In addition, we might perhaps also want to include all kinds of complicated instructions on the input tape, as well as numbers. In order to overcome these difficulties, let us adopt a procedure which I shall refer to as *contraction*, according to which any string of 0s and 1s (with a finite total number of 1s) is *not* simply read as a binary number, but is replaced by a string of 0s, 1s, 2s, 3s, etc., by a prescription whereby each digit of the second sequence is simply the number of 1s lying between successive 0s of the first sequence. For example, the sequence

01000101101010110100011101010111100110

would be replaced

0	1	0	0	0	1	0	1	1	0	1	0	1	1	0	1	0	0	0	1	1	1	0	1	0	1	1	0	0	1	1	0
th	u	s	:	1	0	0	1	2	1	1	2	1	0	0	3	1	1	4	0	2											

We can now read the numbers 2, 3, 4, ... as markers or instructions of some kind. Indeed, let us regard 2 as simply a 'comma', indicating the space between two numbers, whereas 3, 4, 0, ... could, according to our wishes, represent various instructions or notations of interest, such as 'minus sign', 'plus', 'times', 'go to the location with the following number', 'iterate the previous operation the following number of times', etc. We now have various strings of 0s, and 1s which are separated by higher digits. The former are to represent ordinary numbers written in the binary scale. Thus, the above would read (with 'comma' for '2'):

(binary number 1001) comma (binary number 11) comma ...

Using standard Arabic notation '9', '3', '4', '0' for the respective binary numbers 1001, 11, 100, 0, we get, for the entire sequence:

9, 3, 4 (instruction 3) 3 (instruction 4) 0,

In particular, this procedure gives us a means of terminating the description of a number (and thereby distinguishing it from an infinite stretch of blank tape on the right) simply by using a comma at the end. Moreover, it enables us to code any finite sequence of natural numbers, written in the binary notation, as a *single* sequence of 0s and 1s, where we use commas to separate the numbers. Let us see how this works in a specific case. Consider the sequence

5, 13, 0, 1, 1, 4,

for example. In binary notation this is

101, 1101, 0, 1, 1, 100,

which is coded on the tape, by *expansion* (i.e. the inverse of the above contraction procedure), as

... 00001001011010100101100110101101011010
0011000...

To achieve this coding in a simple direct way we can make replacements in our original sequence of binary numbers as follows

0 → 0
1 → 10
, → 110

and then adjoin an unlimited supply of 0s at both ends. It is made clearer how this has been applied to the above tape, if we space it out:

0000 10 0 10 110 10 10 0 10 110 0 110 10 110 10
110 10 0 0 110 00

I shall refer to this notation for (sets of) numbers as the *expanded binary* notation. (So, in particular, the expanded binary form of 13 is 1010010.)

There is one final point that should be made about this coding. It is just a technicality, but necessary for completeness.³ In the binary (or denary) representation of natural numbers there is a slight redundancy in that 0s placed on the far left of an expression do not 'count' – and are normally omitted, e.g. 00110010 is the same binary number as 110010 (and 0050 is the same denary number as 50). This redundancy extends to the number zero itself, which can be written 000 or 00 just as well as 0. Indeed a blank space should, logically, denote zero as well! In ordinary notation that would lead to great confusion, but it fits in well with the notation just described above. Thus a zero between two commas can just as well be written as two commas next to one another (,,) which would be coded on the tape as two pairs 11 separated by a single 0:

... 001101100...

Thus the above set of six numbers can also be written in binary notation as

101,1101,,1,1,100,

and coded on the tape, in expanded binary form, as

... 000010010110101001011011010110101101000
11000...

(which has one 0 missing from the sequence that we had before).

We can now consider a Turing machine for effecting, say, Euclid's algorithm, applying it to pairs of numbers written in the expanded binary notation. For example, for the pair of numbers 6, 8 that we considered earlier, instead of using

... 00000000000111110111111100000 ...,

as we did before, we consider the binary representations of 6 and 8, namely 110 and 1000, respectively. The *pair* is

6,8, i.e., in binary notation, 110,1000,

which, by expansion, is coded as the tape

... 00000101001101000011000000 ...

For this particular pair of numbers there is no gain in conciseness from the unary form. Suppose, however, that we take, say, the (denary) numbers 1583169 and 8610. In binary notation these would be

110000010100001000001, 10000110100010,

so we have the pair coded as the tape

... 001010000001001000001000000101101000001
010010000100110...

which all fits on two lines, whereas in the unary notation, the tape representing '1583169, 8610' would more than fill this entire book!

A Turing machine that effects Euclid's algorithm when the numbers are expressed in expanded binary notation could, if desired, be obtained simply by adjoining to EUC a suitable pair of subroutine algorithms which translate between unary and expanded binary. This would actually be extremely inefficient, however, since the inefficiency of the unary numbering system would still be 'internally' present and would show up in the

slowness of the device and in the inordinate amount of external 'rough paper' (which would be on the left-hand part of the tape) that would be needed. A more efficient Turing machine for Euclid's algorithm operating entirely within expanded binary can also be given, but it would not be particularly illuminating for us here.

Instead, in order to illustrate how a Turing machine can be made to operate on expanded binary numbers, let us try something a good deal simpler than Euclid's algorithm, namely the process of simply *adding one* to a natural number. This can be effected by the Turing machine (which I shall call $XN + 1$):

$00 \rightarrow 00R, 01 \rightarrow 11R, 10 \rightarrow 00R, 11 \rightarrow 101R,$
 $100 \rightarrow 110L, 101 \rightarrow 101R, 110 \rightarrow 01STOP, 111 \rightarrow 1000L,$
 $1000 \rightarrow 1011L, 1001 \rightarrow 1001L, 1010 \rightarrow 110OR,$
 $1011 \rightarrow 101R, 1101 \rightarrow 1111R, 1110 \rightarrow 111R,$
 $1111 \rightarrow 1110R.$

Again, some dedicated readers might care to check that this Turing machine actually does what it is supposed to do, by applying it to, say, the number 167, which has binary representation 10100111 and so would be given by the tape

... 0000100100010101011000 ...

To add one to a binary number, we simply locate the final 0 and change it to 1 and then replace all the 1s which follow by 0s, e.g. $167 + 1 = 168$ is written in binary notation as

$10100111 + 1 = 10101000.$

Thus our 'adding-one' Turing machine should replace the aforementioned tape by

... 0000100100100001100000 ...

which indeed it does.

Note that even the very elementary operation of simply adding one is a bit complicated with this notation, using fifteen instructions and eight different internal states! Things were a lot simpler with the unary notation, of course, since 'adding one' then simply means extending the string of 1s by one further 1, so it is not

surprising that our machine $UN + 1$ was more basic. However, for very large numbers, $UN + 1$ would be exceedingly slow because of the inordinate length of tape required, and the more complicated machine $XN + 1$, which operates with the more compact expanded binary notation, would be better.

As an aside, I point out an operation for which the Turing machine actually looks simpler for expanded binary than for unary notation, namely *multiplying by two*. Here, the Turing machine $XN \times 2$, given by

$00 \rightarrow 00R, 01 \rightarrow 10R, 10 \rightarrow 01R, 11 \rightarrow 100R,$
 $100 \rightarrow 111R, 110 \rightarrow 01STOP,$

achieves this in expanded binary, whereas the corresponding machine in unary notation, $UN \times 2$, which was described earlier, is a good deal more complicated!

This gives us some idea of what Turing machines can do at a very basic level. As might be expected, they can, and do, get vastly more complicated than this when operations of some complexity are to be performed. What is the ultimate scope of such devices? Let us consider this question next.

THE CHURCH-TURING THESIS

Once one has gained some familiarity with constructing simple Turing machines, it becomes easy to satisfy oneself that the various basic arithmetical operations, such as adding two numbers together, or multiplying them, or raising one number to the power of another, can indeed all be effected by specific Turing machines. It would not be too cumbersome to give such machines explicitly, but I shall not bother to do this here. Operations where the result is a *pair* of natural numbers, such as division with a remainder, can also be provided – or where the result is an arbitrarily large finite set of numbers. Moreover Turing machines can be constructed for which it is not specified ahead of time which arithmetical operation it is that needs to be performed, but the instructions for this are fed in on the tape. Perhaps the

particular operation that has to be performed depends, at some stage, upon the result of some calculation that the machine has had to perform at some earlier stage. ('If the answer to that calculation was greater than so-and-so, do this; otherwise, do that.') Once it is appreciated that one can make Turing machines which perform arithmetic or simple logical operations, it becomes easier to imagine how they can be made to perform more complicated tasks of an algorithmic nature. After one has played with such things for a while, one is easily reassured that a machine of this type can indeed be made to perform *any mechanical operation whatever*! Mathematically, it becomes reasonable to *define* a mechanical operation to be one that can be carried out by such a machine. The noun 'algorithm' and the adjectives 'computable', 'recursive', and 'effective' are all used by mathematicians to denote the mechanical operations that can be performed by theoretical machines of this type – the Turing machines. So long as a procedure is sufficiently clear-cut and mechanical, then it is reasonable to believe that a Turing machine can indeed be found to perform it. This, after all, was the whole point of our (i.e. Turing's) introductory discussion motivating the very concept of a Turing machine.

On the other hand, it still could be felt that the design of these machines was perhaps unnecessarily restrictive. Allowing the device to read only one binary digit (0 or 1) at a time, and to move only one space at a time along only a *single* one-dimensional tape seems at first sight to be limiting. Why not allow four or five, or perhaps one thousand separate tapes, with a great number of interconnected reading devices running all at once? Why not allow a whole plane of squares of 0s and 1s (or perhaps a three-dimensional array) rather than insisting on a one-dimensional tape? Why not allow other symbols from some more complicated numbering system or alphabet? In fact, none of these changes makes the slightest difference to what can be in principle achieved, though some make a certain amount of difference to the economy of the operations (as would certainly be the case if we allowed more than one tape). The class of operations performed, and thus come under the heading of 'algorithms' (or 'computations' or 'effective procedures' or 'recursive operations'), would

be precisely the same as before even if we broadened the definition of our machines in all these ways at once!

We can see that there is no *necessity* to have more than one tape, so long as the device can keep finding new space on the given tape, as required. For this, it may need to keep shunting data from one place to another on the tape. This may be 'inefficient', but it does not limit what can be in principle achieved.⁴ Likewise, using more than one Turing device in *parallel action* – which is an idea that has become fashionable in recent years, in connection with attempts to model human brains more closely – does *not* in principle gain anything (though there may be an improved speed of action under certain circumstances). Having two separate devices which do not directly communicate with one another achieves no more than having two which *do* communicate; and *if* they communicate, then, in effect, they are just a single device!

What about Turing's restriction to a one-dimensional tape? If we think of this tape as representing the 'environment', we might prefer to think of it as a planar surface rather than as a one-dimensional tape, or perhaps as a three-dimensional space. A planar surface might seem to be closer to what is needed for a 'flow chart' (as in the above description of the operation of Euclid's algorithm) than a one-dimensional tape would be.* There is, however, no difficulty in principle about writing out the operation of a flow diagram in a 'one-dimensional' form (e.g. by the use of an ordinary verbal description of the chart). The two-dimensional planar display is only for our own convenience and ease of comprehension and it makes no difference to what can in principle be achieved. It is always possible to code the location of a mark or an object on a two-dimensional plane, or even in a three-dimensional space, in a straightforward way on a one-dimensional tape. (In fact, using a two-dimensional plane is

* As things have been described here, this flow chart itself would actually be part of the 'device' rather than of the external environment 'tape'. It was the actual numbers A , B , $A - B$, etc., which we represented on the tape. However, we shall be wanting also to express the specification of the *device* in a linear one-dimensional form. As we shall see later, in connection with the *universal* Turing machine, there is an intimate relation between the specification for a particular 'device' and the specification of possible 'data' (or 'program') for a given device. It is therefore convenient to have *both* of these in one-dimensional form.

completely equivalent to using *two* tapes. The two tapes would supply the two 'coordinates' that would be needed for specifying a point on a two-dimensional plane; likewise *three* tapes can act as 'coordinates' for a point in a three-dimensional space.) Again this one-dimensional coding may be 'inefficient', but it does not limit what can be achieved in principle.

Despite all of this, we might still question whether the concept of a Turing machine really does incorporate *every* logical or mathematical operation that we would wish to call 'mechanical'. At the time that Turing wrote his seminal paper, this was considerably less clear than it is today, so Turing found it necessary to put his case in appreciable detail. Turing's closely argued case found additional support from the fact that, quite independently (and actually a little earlier), the American logician Alonzo Church (with the help of S. C. Kleene) had put forward a scheme – the lambda calculus – also aimed at resolving Hilbert's *Entscheidungsproblem*. Though it was much less obviously a fully comprehensive mechanical scheme than was Turing's, it had some advantages in the striking economy of its mathematical structure. (I shall be describing Church's remarkable calculus at the end of this chapter.) Also independently of Turing there were yet other proposals for resolving Hilbert's problem (see Gandy 1988), most particularly that of the Polish-American logician Emil Post (a little later than Turing, but with ideas considerably more like those of Turing than of Church). All these schemes were soon shown to be completely equivalent. This added a good deal of force to the viewpoint, which became known as the *Church–Turing Thesis*, that the Turing machine concept (or equivalent) actually does define what, mathematically, we mean by an algorithmic (or effective or recursive or mechanical) procedure. Now that high-speed electronic computers have become such a familiar part of our lives, not many people seem to feel the need to question this thesis in its original form. Instead, some attention has been turned to the matter of whether actual *physical* systems (presumably including human brains) – subject as they are to precise *physical* laws – are able to perform more than, less than, or precisely the same logical and mathematical operations as Turing machines. For my own part, I am very happy to accept the original *math-*

ematical form of the Church–Turing Thesis. Its relation to the behaviour of actual physical systems, on the other hand, is a separate issue which will be a major concern for us later in this book.

NUMBERS OTHER THAN NATURAL NUMBERS

In the discussion given above, we considered operations on *natural numbers*, and we noted the remarkable fact that single Turing machines can handle natural numbers of arbitrarily large size, despite the fact that each machine has a fixed *finite* number of distinct internal states. However, one often needs to work with more complicated kinds of number than this, such as negative numbers, fractions, or infinite decimals. Negative numbers and fractions (e.g. numbers like $-597/26$) can be easily handled by Turing machines, and the numerators and denominators can be as large as we like. All we need is some suitable coding for the signs '-' and '/', and this can easily be done using the expanded binary notation described earlier (for example, '3' for '-' and '4' for '/' – coded as 1110 and 11110, respectively, in the expanded binary notation). Negative numbers and fractions are thus handled in terms of finite sets of natural numbers, so with regard to general questions of computability they give us nothing new.

Likewise, *finite* decimal expressions of unrestricted length give us nothing new, since these are just particular cases of fractions. For example, the finite decimal approximation to the irrational number π , given by 3.14159265, is simply the fraction $314159265/100000000$. However, *infinite* decimal expressions, such as the *full* non-terminating expansion

$$\pi = 3.14159265358979 \dots$$

present certain difficulties. Neither the input nor the output of a Turing machine can, strictly speaking, be an infinite decimal. One might think that we could find a Turing machine to churn out *all* the successive digits, 3, 1, 4, 1, 5, 9, . . . , of the above expansion for π one after the other on the output tape, where we simply allow the machine to run on forever. But this is *not allowed* for a Turing

machine. We must wait for the machine to halt (indicated by the bell ringing!) before we are allowed to examine the output. So long as the machine has not reached a STOP order, the output is subject to possible change and so cannot be trusted. After it has reached STOP, on the other hand, the output is necessarily finite.

There is, however, a procedure for *legitimately* making a Turing machine produce digits one after the other, in a way very similar to this. If we wish to generate an infinite decimal expansion, say that of π , we could have a Turing machine produce the whole-number part, 3, by making the machine act on 0, then we could produce the first decimal digit, 1, by making the machine act on 1, then the second decimal digit, 4, by making it act on 2, then the third, 1, by making it act on 3, and so on. In fact a Turing machine for producing the entire decimal expansion of π in *this* sense certainly *does* exist, though it would be a little complicated to work it out explicitly. A similar remark applies to many other irrational numbers, such as $\sqrt{2} = 1.414213562 \dots$. It turns out, however, that some irrationals (remarkably) cannot be produced by any Turing machine at all, as we shall see in the next chapter. The numbers that *can* be generated in this way are called *computable* (Turing 1937). Those that cannot (actually the vast majority!) are *non-computable*. I shall come back to this matter, and related issues, in later chapters. It will have some relevance for us in relation to the question of whether an *actual physical object* (e.g. a human brain) can, according to our physical theories, be adequately described in terms of computable mathematical structures.

The issue of computability is an important one generally in mathematics. One should not think of it as a matter which applies just to *numbers* as such. One can have Turing machines which operate directly on *mathematical formulae*, such as algebraic or trigonometric expressions, for example, or which carry through the formal manipulations of the calculus. All that one needs is some form of precise coding into sequences of 0s and 1s, of all the mathematical symbols that are involved, and then the Turing machine concept can be applied. This, after all, was what Turing had in mind in his attack on the *Entscheidungsproblem*, which asks for an algorithmic procedure for answering mathematical

questions of a *general* nature. We shall be coming back to this shortly.

THE UNIVERSAL TURING MACHINE

I have not yet described the concept of a *universal* Turing machine. The principle behind this is not too difficult to give, even though the details are complicated. The basic idea is to code the list of instructions for an arbitrary Turing machine T into a string of 0s and 1s that can be represented on a tape. This tape is then used as the initial part of the input for some *particular* Turing machine U – called a universal Turing machine – which then acts on the remainder of the input just as T would have done. The universal Turing machine is a universal mimic. The initial part of the tape gives the universal machine U the full information that it needs for it to imitate any given machine T exactly!

To see how this works we first need a systematic way of *numbering* Turing machines. Consider the list of instructions defining some particular Turing machine, say one of those described above. We must code this list into a string of 0s and 1s according to some precise scheme. This can be done with the aid of the 'contraction' procedure that we adopted before. For if we represent the respective symbols R, L, STOP, the arrow ($\rightarrow$), and the comma as, say, the numerals 2, 3, 4, 5, and 6, we can code them as contractions, by 110, 1110, 11110, 111110, and 1111110. Then the digits 0 and 1, coded as 0 and 10, respectively, can be used for the actual strings of these symbols appearing in the table. We do not need to have a different notation to distinguish the large figure 0 and 1 in the Turing machine table from the smaller boldface ones, since the position of the large digits at the end of the binary numbering is sufficient to distinguish them from the others. Thus, for example, 1101 would be read as the binary number 1101 and coded on the tape as 1010010. In particular, 00 would be read as 00, which can, without ambiguity, be coded 0, or as a symbol omitted altogether. We can economize considerably by not actually bothering to code any arrow nor any of the symbols immediately preceding it, relying instead upon the numerical ordering of instructions to specify what those symbols must be –

although to adopt this procedure we must make sure that there are no gaps in this ordering, supplying a few extra 'dummy' orders where required. (For example, the Turing machine $XN + 1$ has no order telling us what to do with 1100, since this combination never occurs in the running of the machine, so we must insert a 'dummy' order, say $1100 \rightarrow 00R$, which can be incorporated into the list without changing anything. Similarly we should insert $101 \rightarrow 00R$ into the machine $XN \times 2$.) Without such 'dummies', the coding of the subsequent orders in the list would be spoiled. We do not actually need the comma at the end of each instruction, as it turns out, since the symbols L or R suffice to separate the instructions from one another. We therefore simply adopt the following coding:

0 for 0 or 0, 10 for 1 or 1, 110 for R, 1110 for L,
 11110 for STOP

As an example, let us code the Turing machine $XN + 1$ (with the $1100 \rightarrow 00R$ instruction inserted). Leaving out the arrows, the digits immediately preceding them, and also the commas, we have

00R 11R 00R 101R 110L 101R 01STOP 1000L 1011L
1001L 1100R 101R 00R 1111R 111R 1110R.

We can improve on this by leaving out every 00 and replacing each 01, by simply 1, in accordance with what has been said earlier, to get

R11RR101R110L101R1STOP1000L1011L1001L1100R101RR1111R
111R1110R.

This is coded as the tape sequence

11010101101101001011010100111010010110101
11101000011101001010111010001011101010001
10100101101101010101011010101011010101001
10.

As two further minor economies, we may as well always delete the initial 110 (together with the infinite stretch of blank tape that precedes it) since this denotes 00R, which represents the initial instruction $00 \rightarrow 00R$ that I have been implicitly taking to be common to *all* Turing machines – so that the device can start

arbitrarily far to the left of the marks on the tape and run to the right until it comes up to the first mark – and we may as well always delete the final 110 (and the implicit infinite sequence of 0s which is assumed to follow it) since all Turing machines must have their descriptions ending this way (because they all end with R, L, or STOP). The resulting *binary number* is the *number* of the Turing machine, which in the case of $XN + 1$ is:

10101101101001011010100111010010110101111
01000011101001010111010001011101010001101
0010110110101010101101010101101010100.

In standard denary notation, this particular number is

450813704461563958982113775643437908.

We sometimes loosely refer to the Turing machine whose number is n as the n th Turing machine, denoted T_n . Thus $XN + 1$ is the 450813704461563958982113775643437908th Turing machine!

It is a striking fact that we appear to have to go this far along the 'list' of Turing machines before we find one that even performs so trivial an operation as adding one (in the expanded binary notation) to a natural number! (I do not think that I have been grossly inefficient in my coding, though I can see room for some minor improvements.) Actually there are some Turing machines with smaller numbers which are of interest. For example, $UN + 1$ has the binary number

101011010111101010

which is merely 177642 in denary notation! Thus the particularly trivial Turing machine $UN + 1$, which merely places an additional 1 at the end of a sequence of 1s, is the 177642nd Turing machine. For curiosity's sake, we may note that 'multiplying by two' comes somewhere between these two in the list of Turing machines, in either notation, for we find that the number for $XN \times 2$ is 10389728107 while that of $UN \times 2$ is 1492923420919872026917547669.

It is perhaps not surprising to learn, in view of the sizes of these numbers, that the vast majority of natural numbers do not give

working Turing machines at all. Let us list the first thirteen Turing machines according to this numbering:

T_0 :	$00 \rightarrow 00R, 01 \rightarrow 00R,$
T_1 :	$00 \rightarrow 00R, 01 \rightarrow 00L,$
T_2 :	$00 \rightarrow 00R, 01 \rightarrow 01R,$
T_3 :	$00 \rightarrow 00R, 01 \rightarrow 00STOP,$
T_4 :	$00 \rightarrow 00R, 01 \rightarrow 10R,$
T_5 :	$00 \rightarrow 00R, 01 \rightarrow 01L,$
T_6 :	$00 \rightarrow 00R, 01 \rightarrow 00R, 10 \rightarrow 00R,$
T_7 :	$00 \rightarrow 00R, 01 \rightarrow ???,$
T_8 :	$00 \rightarrow 00R, 01 \rightarrow 100R,$
T_9 :	$00 \rightarrow 00R, 01 \rightarrow 10L,$
T_{10} :	$00 \rightarrow 00R, 01 \rightarrow 11R,$
T_{11} :	$00 \rightarrow 00R, 01 \rightarrow 01STOP,$
T_{12} :	$00 \rightarrow 00R, 01 \rightarrow 00R, 10 \rightarrow 00R.$

Of these, T_0 simply moves on to the right obliterating everything that it encounters, never stopping and never turning back. The machine T_1 ultimately achieves the same effect, but in a clumsier way, jerking backwards after it obliterates each mark on the tape. Like T_0 , the machine T_2 also moves on endlessly to the right, but is more respectful, simply leaving everything on the tape just as it was before. None of these is any good as a Turing machine since none of them ever stops. T_3 is the first respectable machine. It indeed stops, modestly, after changing the first (leftmost) 1 into a 0.

T_4 encounters a serious problem. After it finds its first 1 on the tape it enters an internal state for which there is no listing, so it has no instructions as to what to do next. T_8 , T_9 , and T_{10} encounter the same problem. The difficulty with T_7 is even more basic. The string of 0s and 1s which codes it involves a sequence of *five* successive 1s: 110111110. There is no interpretation for such a sequence, so T_7 will get stuck as soon as it finds its first 1 on the tape. (I shall refer to T_7 , or any other machine T_n for which the binary expansion of n contains a sequence of more than four 1s as being *not correctly specified*.) The machines T_5 , T_6 , and T_{12} encounter problems similar to those of T_0 , T_1 , and T_2 . They simply run on indefinitely without ever stopping. All of the

machines T_0 , T_1 , T_2 , T_4 , T_5 , T_6 , T_7 , T_8 , T_9 , T_{10} , and T_{12} are duds! Only T_3 and T_{11} are working Turing machines, and not very interesting ones at that. T_{11} is even more modest than T_3 . It stops at its first encounter with a 1 and it doesn't change a thing!

We should note that there is also a redundancy in our list. The machine T_{12} is identical with T_6 , and also identical in action with T_0 , since the internal state 1 of T_6 and T_{12} is never entered. We need not be disturbed by this redundancy, nor by the proliferation of dud Turing machines in the list. It would indeed be possible to improve our coding so that a good many of the duds are removed and the redundancy considerably reduced. All this would be at the expense of complicating our poor universal Turing machine which has to decipher the code and pretend to be the Turing machine T_n whose number n it is reading. This might be worth doing if we could remove *all* the duds (or the redundancy). But this is *not* possible, as we shall see shortly! So let us leave our coding as it is.

It will be convenient to interpret a tape with its succession of marks, e.g.

... 0001101110010000 ...

as the binary representation of some number. Recall that the 0s continue indefinitely at both ends, but that there is only a finite number of 1s. I am also assuming that the number of 1s is *non-zero* (i.e. there is at least one 1). We could choose to read the finite string of symbols between the first and last 1 (inclusive), which in the above case is

110111001,

as the binary description of a natural number (here 441, in denary notation). However, this procedure would only give us *odd* numbers (numbers whose binary representation ends in a 1) and we want to be able to represent *all* natural numbers. Thus we adopt the simple expedient of removing the final 1 (which is taken to be just a marker indicating the termination of the expression) and reading what is left as a binary number.⁵ Thus, for the above example, we have the binary number

11011100,

which, in denary notation, is 220. This procedure has the advantage that zero is also represented as a marked tape, namely

... 0000001000000 ...

Let us consider the action of the Turing machine T_n on some (finite) string of 0s and 1s on a tape which we feed in on the right. It will be convenient to regard this string also as the binary representation of some number, say m , according to the scheme given above. Let us assume that after a succession of steps the machine T_n finally comes to a halt (i.e. reaches STOP). The string of binary digits that the machine has now produced at the left is the answer to the calculation. Let us also read this as the binary representation of a number in the same way, say p . We shall write this relation, which expresses the fact that when the n th Turing machine acts on m it produces p , as:

$$T_n(m) = p.$$

Now let us look at this relation in a slightly different way. We think of it as expressing one particular operation which is applied to the *pair* of numbers n and m in order to produce the number p . (Thus: given the *two* numbers n and m , we can work out from them what p is by seeing what the n th Turing machine does to m .) This particular operation is an entirely algorithmic procedure. It can therefore be carried out by *one particular* Turing machine U ; that is, U acts on the *pair* (n, m) to produce p . Since the machine U has to act on *both* of n and m to produce the single result p , we need some way of coding the pair (n, m) on the *one* tape. For this, we can assume that n is written out in ordinary binary notation and then immediately terminated by the sequence 111110. (Recall that the binary number of every correctly specified Turing machine is a sequence made up just of 0s, 10s, 110s, 1110s, and 11110s, and it therefore contains no sequence of more than four 1s. Thus if T_n is a correctly specified machine, the occurrence of 111110 indeed signifies that the description of the number n is finished with.) Everything following it is to be simply the tape represented by m according to our above prescription (i.e. the binary number m immediately followed by 1000 ...). Thus this second part is simply the tape that T_n is supposed to act on.

As an example, if we take $n = 11$ and $m = 6$, we have, for the tape on which U has to act, the sequence of marks

... 000101111111011010000 ...

This is made up as follows:

... 0000 (initial blank tape)
1011 (binary representation of 11)
111110 (terminates n)
110 (binary representation of 6)
10000 ... (remainder of tape)

What the Turing machine U would have to do, at each successive step of the operation of T_n on m , would be to examine the structure of the succession of digits in the expression for n so that the appropriate replacement in the digits for m (i.e. T_n 's 'tape') can be made. In fact it is not difficult in principle (though decidedly tedious in practice) to see how one might actually construct such a machine. Its own list of instructions would simply be providing a means of reading the appropriate entry in that 'list' which is encoded in the number n , at each stage of application to the digits of the 'tape', as given by m . There would admittedly be a lot of dodging backwards and forwards between the digits of m and those of n , and the procedure would tend to be exceedingly slow. Nevertheless, a list of instructions for such a machine can certainly be provided; and we call such a machine a *universal* Turing machine. Denoting the action of this machine on the pair of numbers n and m by $U(n, m)$, we have:

$$U(n, m) = T_n(m)$$

for each (n, m) for which T_n is a correctly specified Turing machine.⁶ The machine U , when first fed with the number n , precisely imitates the n th Turing machine!

Since U is a Turing machine, it will itself have a number; i.e. we have

$$U = T_u,$$

for some number u . How big is u ? In fact we can take *precisely*

$n = 724485533533931757719839503961571123795236$
 067255655963110814479660650505940424109031048
 36136323593656444345838222688327876762655614
 469281411771501784255170755408565768975334635
 694247848859704693472573998858228382779529468
 346052106116983594593879188554632644092552550
 582055598945189071653741489603309675302043155
 362503498452983232065158304766414213070881932
 971723415105698026273468642992183817215733348
 282307345371342147505974034518437235959309064
 002432107734217885149276079759763441512307958
 639635449226915947965461471134570014504816733
 756217257346452273105448298078496512698878896
 456976090663420447798902191443793283001949357
 096392170390483327088259620130177372720271862
 591991442827543742235135567513408422229988937
 441053430547104436869587640517812801943753081
 387063994277282315642528923751456544389905278
 079324114482614235728619311833261065612275553
 181020751108533763380603108236167504563585216
 421486954234718742643754442879006248582709124
 042207653875426445413345174856629157429990950
 262300973373813772416217274772361020678685400
 289356608569682262014198248621698902609130940
 298570600174300670086896759034473417412787425
 581201549366393899690581773859165405535670409
 282133222163141097871081459978669599704509681
 841906299443656015145490488092208448003482249
 207730403043188429899393135266882349662101947
 161910701461968523192847482034495897709553561
 107027581748733327296678998798473284098190764
 851272631001740166787363477605857245036964434
 897992034489997455662402937487668839751404451
 665707750060513883991668814072545544665222050
 724262392379211525318162512536305093172863142
 200406457130527580230766518335199588913974813
 7504926429605010013651980186945639498

(or some other possibility of at least that kind of size). This number no doubt seems alarmingly large! Indeed it *is* alarmingly large, but I have not been able to see how it could have been made significantly smaller. The coding procedures and specifications that I have given for Turing machines are quite reasonable and simple ones, yet one is inevitably led to a number of this kind of size for the coding of an actual universal Turing machine.⁷

I have said that all modern general purpose computers are, in effect, universal Turing machines. I do not mean to imply that the logical design of such computers need resemble at all closely the kind of description for a universal Turing machine that I have just given. The point is simply that, by supplying any universal Turing machine first with an appropriate program (initial part of the input tape), it can be made to mimic the behaviour of any Turing machine whatever! In the description above, the program simply takes the form of a single number (the number n), but other procedures are possible, there being many variations on Turing's original theme. In fact in my own descriptions I have deviated somewhat from those that Turing originally gave. None of these differences is important for our present needs.

THE INSOLUBILITY OF HILBERT'S PROBLEM

We now come to the purpose for which Turing originally put forward his ideas, the resolution of Hilbert's broad-ranging *Entscheidungsproblem*: is there some mechanical procedure for answering all mathematical problems, belonging to some broad, but well-defined class? Turing found that he could phrase his version of the question in terms of the problem of deciding whether or not the n th Turing machine would actually ever *stop* when acting on the number m . This problem was referred to as the *halting problem*. It is an easy matter to construct an instruction list for which the machine will not stop for *any* number m (for example, $n = 1$ or 2 , as given above, or any other case where there are no STOP instructions whatever). Also there are many instruction lists for which the machine would always stop, whatever number it is given (e.g. $n = 11$); and some machines would stop for some numbers but not for others. One could fairly say that a